Building a RAG Document Assistant with Postgres and pgvector

A detailed AI Data Platform Architecture article on document ingestion, chunking, embeddings, vector search, prompt grounding, and retrieval quality.

Introduction

Retrieval-augmented generation, or RAG, is one of the most practical patterns in modern AI data platforms. It gives language models access to domain-specific information without requiring every fact to live inside the model weights. Instead of asking a model to answer from memory, the platform retrieves relevant source material, injects that context into a prompt, and asks the model to answer from the retrieved evidence.

From an architecture perspective, RAG is not just a prompt engineering technique. It is a data architecture pattern. It involves document ingestion, parsing, chunking, embeddings, vector search, metadata management, retrieval quality evaluation, prompt construction, source citation, and operational reliability.

The `04-rag-document-assistant` project is a local implementation of that pattern. It builds a Markdown/PDF document assistant using FastAPI, Postgres, `pgvector`, a document ingestion pipeline, chunking logic, embeddings, vector search, and a source-grounded `/ask` endpoint.

The goal is simple: turn local documents into searchable, retrievable context that can support grounded answers.

Why This Project Matters

Most AI applications eventually run into the same limitation: the model does not automatically know your private documents, project notes, technical specs, data contracts, architecture diagrams, runbooks, or internal knowledge base.

RAG solves this by creating a bridge between enterprise data and language model reasoning.

In an AI data platform, that bridge needs to be designed carefully. It is not enough to throw documents into a vector database and hope the model answers correctly. A strong RAG system needs to answer several architectural questions:

- How are documents loaded and normalized?
- How are documents split into chunks?
- Which embedding model creates the vector representation?
- Where are embeddings stored?
- How is similarity search performed?
- How are retrieved chunks filtered and ranked?
- How is source context inserted into the prompt?
- How does the system know when it does not have enough context?
- How is retrieval quality evaluated over time?

This project addresses each of those questions in a local, inspectable implementation.

System Overview

The project uses a FastAPI service backed by Postgres and `pgvector`.

At a high level, the system works like this:

```
Document
-> load Markdown/text/PDF
-> normalize content
-> split into overlapping chunks
-> generate embeddings
-> store documents and chunks in Postgres + pgvector

User question
-> embed query
-> search pgvector
-> retrieve relevant chunks
-> build source-grounded prompt
-> optionally call local LLM API
-> return answer with sources
```

The key components are:

```
| Component | Role |
| ------------------- |
------------------------------------------------------------------------- |
| FastAPI API | Provides ingestion, search, document listing, health, and RAG endpoints |
| Postgres | Stores document metadata, chunk text, source paths, timestamps, and hashes |
| pgvector | Stores and searches embedding vectors using cosine distance |
| Document loader | Reads Markdown, text, and PDF documents |
| Chunker | Splits text into overlapping chunks |
| Embedding provider | Creates vectors using a hash-based provider or optional Ollama
embeddings |
| Vector search layer | Searches chunks and returns similarity-ranked context |
| Prompt builder | Formats retrieved chunks into a grounded RAG prompt |
| Optional LLM API | Connects to the Phase 3 local LLM wrapper through `LLM_API_URL` |
```

Document Ingestion

The first step in the RAG pipeline is ingestion.

The project supports Markdown, plain text, and PDF files. When a document is ingested, the system loads the file, extracts text, normalizes whitespace, calculates a SHA-256 content hash, and creates a durable document record.

That content hash matters. In production systems, deduplication is an important part of document management. Without it, the same document can be indexed multiple times, which can pollute retrieval results and create confusing source citations.

The ingestion endpoint is:

```
POST /documents/ingest
```

A request includes a file path, optional title, and optional metadata. After ingestion, the API returns the document ID, source path, title, number of chunks created, and content hash.

From an architecture standpoint, this creates the first important boundary: raw files become normalized, trackable platform records.

Chunking Strategy

After loading a document, the system splits it into chunks.

Chunking is one of the most important design choices in a RAG system. If chunks are too small, retrieval may return isolated fragments without enough surrounding context. If chunks are too large, similarity search becomes less precise because each vector represents too many concepts at once.

This project uses overlapping text chunks with two main settings:

```
| Setting | Default | Purpose |
| --------------- | ---------------: |
------------------------------------------------------------ |
| `CHUNK_SIZE` | `900` characters | Keeps retrieved context focused |
| `CHUNK_OVERLAP` | `150` characters | Reduces the chance of splitting useful context at
boundaries |
```

Overlap is a practical compromise. It helps preserve continuity between chunks, especially when an important idea spans two adjacent sections. But overlap also increases duplicate content, so it must be tuned.

From an AI Data Platform Architect's perspective, chunking is not just preprocessing. It is retrieval design. The quality of generated answers depends heavily on whether the retrieval layer can find chunks that are both specific and complete enough to support the answer.

Embeddings

Once chunks are created, the system embeds each chunk.

Embeddings convert text into vectors that can be compared mathematically. Similar chunks should land near similar queries in vector space. This is what allows the system to search by meaning rather than only by keywords.

The project supports two embedding paths:

```
| Provider | Purpose |
| -------- | ------------------------------------------------------------------ |
| `hash` | Deterministic local smoke-test embeddings with no model dependency |
| `ollama` | Optional semantic embeddings using an Ollama embedding model |
```

The default hash embedding provider is intentionally simple. It makes the system easy to run locally without downloading an embedding model. That is useful for testing the API, database schema, ingestion flow, and vector search mechanics.

For stronger semantic retrieval, the project can use Ollama embeddings:

```
ollama pull nomic-embed-text
EMBEDDING_PROVIDER=ollama
OLLAMA_EMBEDDING_MODEL=nomic-embed-text
EMBEDDING_DIMENSION=768
```

The embedding dimension is important because the `pgvector` column is declared as `vector(n)`. If the embedding model changes dimension, the database schema or volume must be recreated so the stored vector shape matches the model output.

Postgres as the Vector Store

This project uses Postgres with `pgvector` as the vector database.

That is an intentional architectural choice. Dedicated vector databases are powerful, but Postgres plus `pgvector` is an excellent learning and prototyping path because it combines relational metadata and vector similarity search in one system.

The schema has two main tables:

`documents`

Stores one row per source document:

- document ID
- source path
- title
- content hash
- metadata
- created timestamp

`document_chunks`

Stores searchable chunks:

- chunk ID
- parent document ID
- chunk index
- chunk text
- token estimate
- embedding vector
- metadata
- created timestamp

This schema preserves both the semantic search layer and the operational metadata needed to explain where a result came from.

Vector Search

The vector search layer embeds the user's query, compares it against stored chunk embeddings, filters by a similarity threshold, orders by nearest vectors, and returns the top results.

The core retrieval pattern uses cosine distance:

```
SELECT
c.id AS chunk_id,
d.title,
c.chunk_index,
c.content,
1 - (c.embedding <=> :query_embedding) AS similarity
FROM document_chunks c
JOIN documents d ON d.id = c.document_id
WHERE 1 - (c.embedding <=> :query_embedding) >= :similarity_threshold
ORDER BY c.embedding <=> :query_embedding
```

```
LIMIT :limit;
```

The similarity score gives the platform an interpretable signal. It helps determine whether retrieved chunks are close enough to the query to be useful. It also gives engineers a lever for tuning retrieval behavior.

The API endpoint is:

```
POST /search
```

This endpoint is useful on its own because it lets the platform inspect retrieval behavior before introducing answer generation. That separation is important. If a RAG answer is poor, the first debugging question should be: did retrieval find the right context?

The RAG API Endpoint

The project exposes a source-grounded answer endpoint:

```
POST /ask
```

The `/ask` flow is:

1. Embed the user question.
2. Search `pgvector` for relevant chunks.
3. Build a prompt using retrieved source context.
4. Optionally call the Phase 3 local LLM API.
5. Return the answer, sources, and optionally the generated prompt.

If `LLM_API_URL` is not configured, the endpoint still returns retrieved sources and a placeholder answer. That is a useful architecture decision because it lets the retrieval system be tested independently from model generation.

When `LLM_API_URL` is configured, the RAG assistant can send the final prompt to the local LLM wrapper built in Phase 3:

```
04-rag-document-assistant
-> retrieve context from Postgres + pgvector
-> build grounded prompt
-> call 03-local-llm-serving-lab /generate
```

This creates a clean separation between retrieval infrastructure and model-serving infrastructure.

Prompt Design

The project includes a RAG answer template designed around source grounding:

```
You are a source-grounded document assistant.
Answer only from the supplied context.
If the context does not contain the answer, say that the documents do not provide enough
information.
Always cite source titles and chunk numbers when possible.
```

This prompt template matters because RAG systems need guardrails. The model should not freely invent an answer when the retrieved context is weak. It should cite source titles and chunk numbers so the user can trace the answer back to evidence.

The prompt also includes similarity scores. That is useful during debugging because it gives engineers visibility into how strongly each retrieved chunk matched the query.

The project also includes a query rewrite template. Query rewriting is not enabled by default, but it is documented as a future improvement for conversational questions, ambiguous references, and follow-up prompts.

Retrieval Quality

RAG quality depends heavily on retrieval quality.

A retrieval layer is working well when the top chunks are:

- relevant to the question
- specific enough to support the answer
- diverse enough to avoid repeating the same passage
- grounded with source metadata
- ranked in a way that matches human judgment

The project documents several metrics:

```
| Metric | Meaning |
| ---------------------- |
------------------------------------------------------------------- |
| Recall@k | Whether a relevant chunk appears in the top `k` results |
| Precision@k | How many of the top `k` chunks are useful |
| MRR | How high the first relevant chunk appears |
| Similarity score spread | Whether the top result is clearly stronger than lower-ranked
results |
| Citation accuracy | Whether the final answer cites the right sources |
```

These metrics are important because RAG should be evaluated as a system, not just as a model response. The answer quality depends on document parsing, chunking, embeddings, retrieval ranking, prompt construction, and generation.

Failure Modes

The project also documents common RAG failure modes:

```
| Failure | Likely Cause | Fix |
| -------------------- | ---------------------------------------- |
---------------------------------------- |
| No results | Threshold too high or no documents indexed | Lower threshold or ingest
documents |
| Repetitive results | Chunk overlap too high | Lower overlap or add diversity reranking |
| Wrong source cited | Retrieved context is weak | Improve embeddings or add query rewriting
|
| Good chunk ranked low | Chunk too large or query too vague | Tune chunk size or rewrite
query |
| Slow search | Large index without tuning | Tune indexes, add filters, benchmark |
```

From a platform architecture perspective, these failure modes are not edge cases. They are normal operating conditions for RAG systems. A production platform needs observability, evaluation datasets, metadata filters, reranking, and feedback loops to continuously improve retrieval.

Why This Architecture Matters

This project is intentionally local, but the architecture maps directly to production AI data platforms.

In production, the same pattern can support:

- internal knowledge assistants
- document search over technical specs
- policy and governance assistants
- data catalog question answering
- customer support knowledge bases
- analytics documentation search
- runbook and incident response assistants

The value comes from combining structured data platform practices with AI application patterns. Postgres stores durable document and chunk metadata. `pgvector` provides semantic retrieval. FastAPI exposes controlled service boundaries. Prompt templates enforce grounding. Retrieval evaluation gives the team a way to measure quality.

That is why RAG belongs in the AI data platform architecture conversation. It is not just a model feature. It is a pipeline that turns enterprise knowledge into retrievable, auditable, source-grounded context.

Production Improvements

This lab creates the foundation, but a production version would add several improvements:

- semantic embeddings by default rather than hash embeddings
- metadata filters for document type, business domain, owner, freshness, or access policy
- embedding model version tracking
- document versioning and re-indexing workflows
- reranking before prompt construction
- evaluation datasets for retrieval quality
- authorization filters before retrieval
- observability for query latency, hit rate, source usage, and answer quality
- batch ingestion jobs for large document collections
- alerts for stale indexes or degraded retrieval quality

These improvements turn a local RAG assistant into a reliable enterprise AI platform component.

Final Takeaway

The `04-rag-document-assistant` project demonstrates the core architecture of a source-grounded AI assistant.

It loads documents, chunks text, creates embeddings, stores vectors in Postgres with `pgvector`, retrieves relevant context, builds grounded prompts, and optionally connects to a local LLM service for answer generation.

For an AI Data Platform Architect, the most important lesson is that RAG is a data system before it is a model interaction. The quality of the final answer depends on how well the platform manages documents, chunks, embeddings, metadata, retrieval, prompt construction, and evaluation.

This project makes those moving parts visible, local, and understandable.